

ECLIPSE

DAW



9 DE OCTUBRE DE 2026

DIEGO GOMEZ ARCAS

Contenido

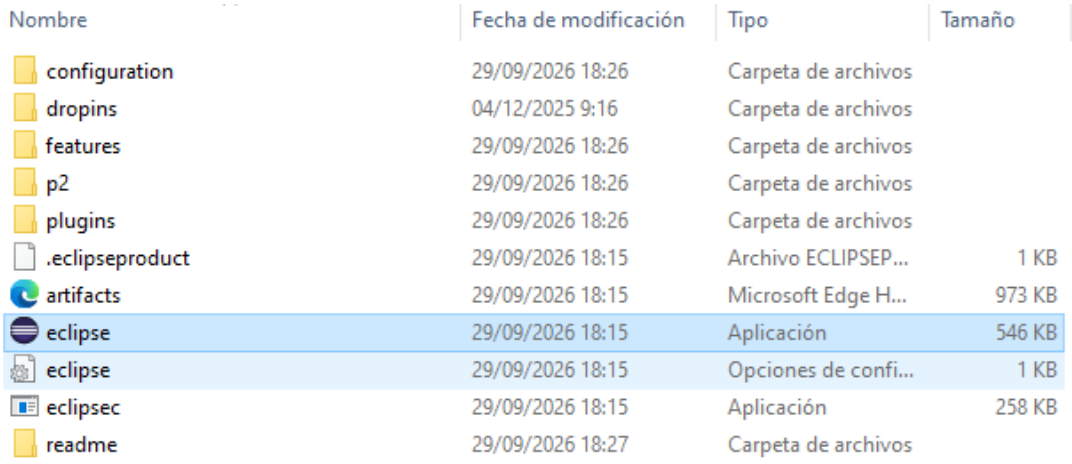
Instalado el jdk 2
Eclipse descargado 2
Arrancamos el tomcat 3
Launcher 3
Definiendo servidor 5
Tomcat 6
Definimos proyecto 7
Codigo 8
Resultados 10
Archivo WAR 12

Instalado el jdk



Comentario: En esta imagen se muestra que el JDK de Java se ha instalado correctamente. El JDK incluye las herramientas necesarias para desarrollar y compilar programas Java.

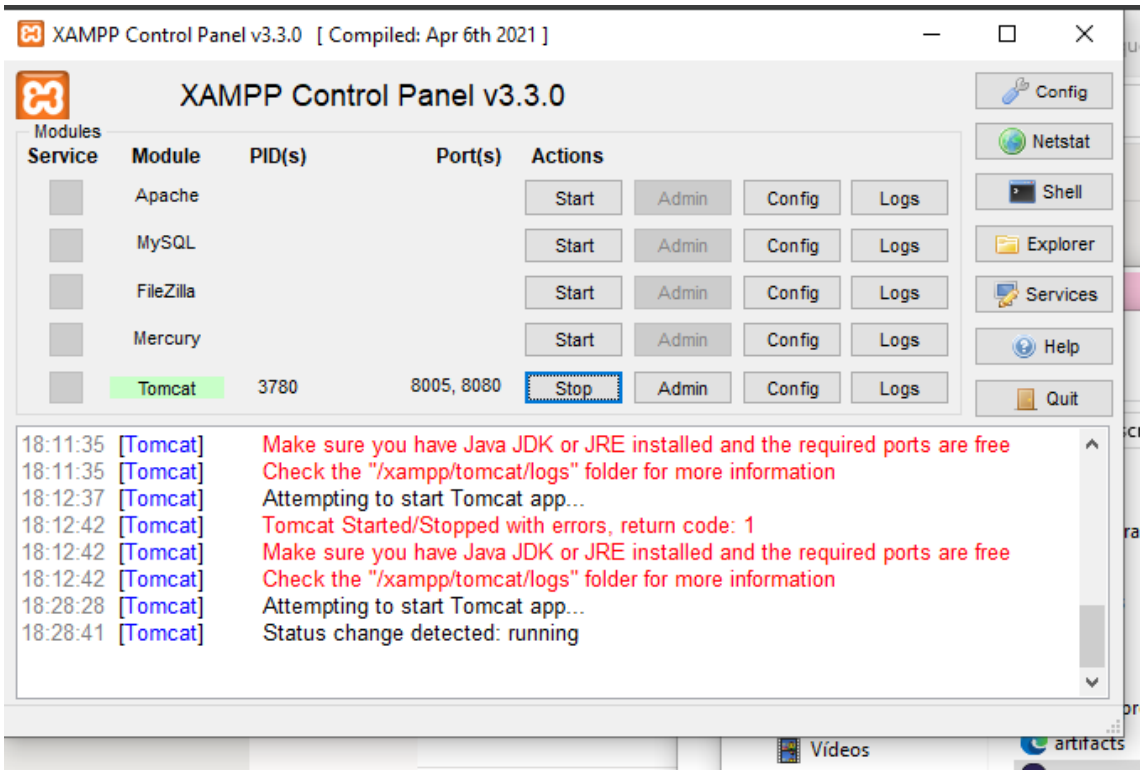
Eclipse descargado



Nombre	Fecha de modificación	Tipo	Tamaño
configuration	29/09/2026 18:26	Carpeta de archivos	
dropins	04/12/2025 9:16	Carpeta de archivos	
features	29/09/2026 18:26	Carpeta de archivos	
p2	29/09/2026 18:26	Carpeta de archivos	
plugins	29/09/2026 18:26	Carpeta de archivos	
.eclipseproduct	29/09/2026 18:15	Archivo ECLIPSEP...	1 KB
artifacts	29/09/2026 18:15	Microsoft Edge H...	973 KB
eclipse	29/09/2026 18:15	Aplicación	546 KB
eclipse	29/09/2026 18:15	Opciones de confi...	1 KB
eclipsec	29/09/2026 18:15	Aplicación	258 KB
readme	29/09/2026 18:27	Carpeta de archivos	

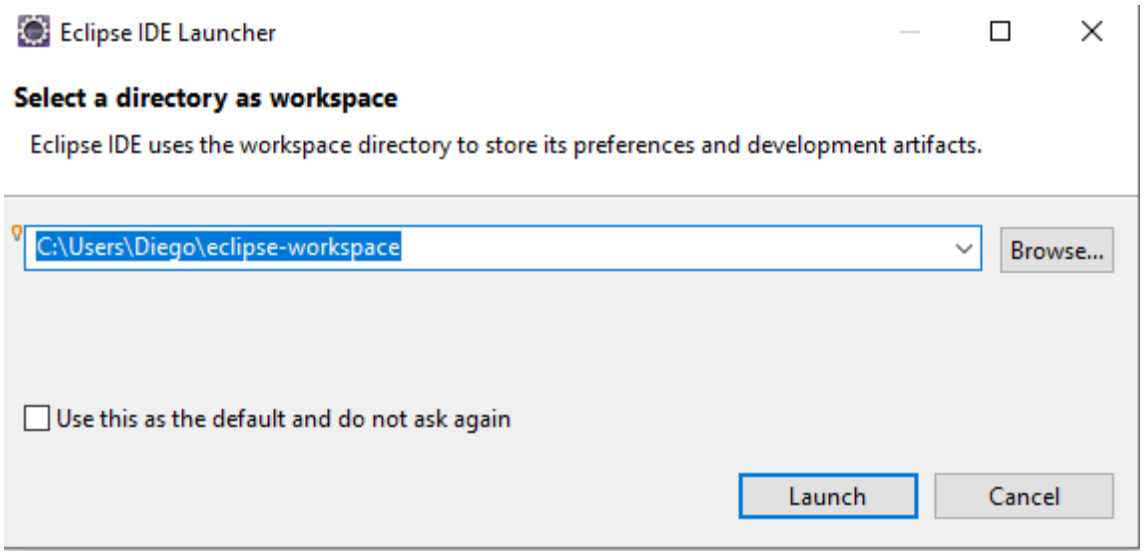
Comentario: Aquí aparecen los archivos de Eclipse descargados o descomprimidos. Desde esta carpeta se puede localizar y ejecutar el entorno de desarrollo.

Arrancamos el tomcat

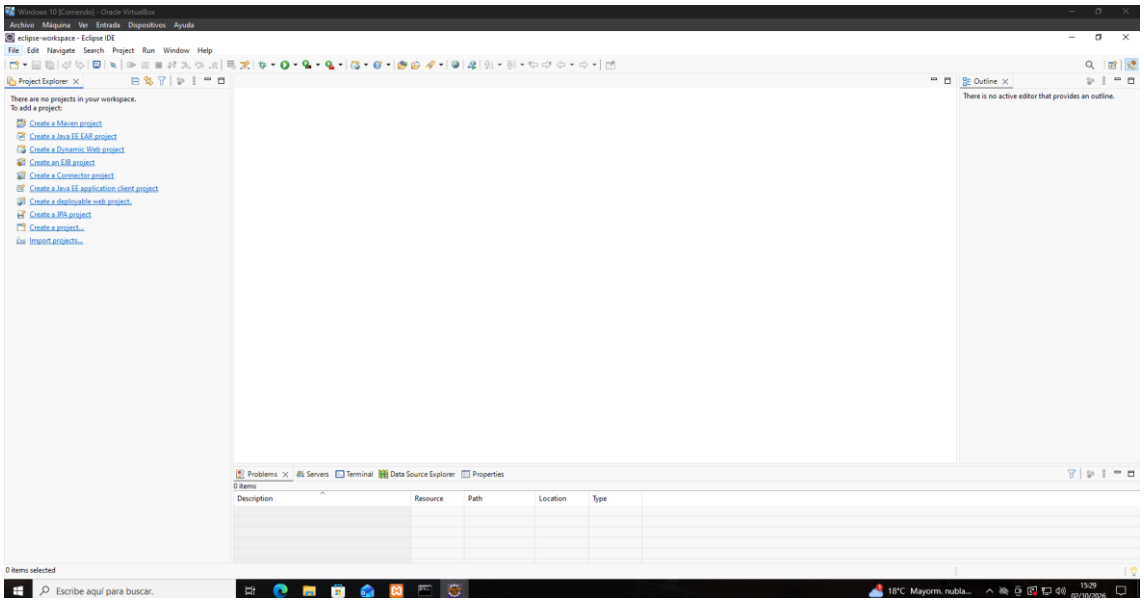


Comentario: En este paso se abre XAMPP y se inicia el servicio Tomcat. Tomcat permite ejecutar aplicaciones web Java en un servidor local.

Launcher

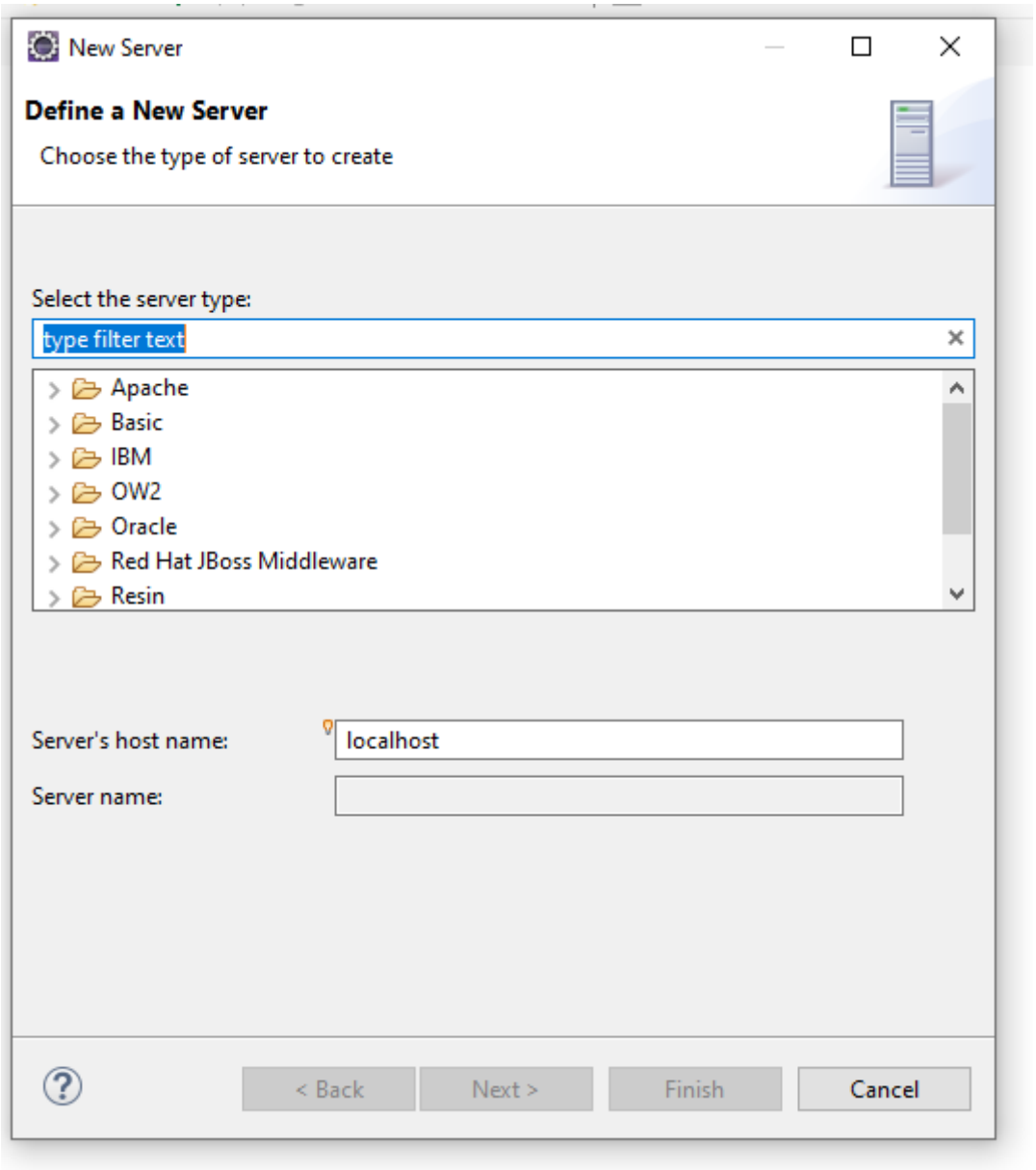


Comentario: Eclipse solicita elegir una carpeta de workspace. Esta carpeta será el espacio de trabajo donde se guardarán los proyectos.



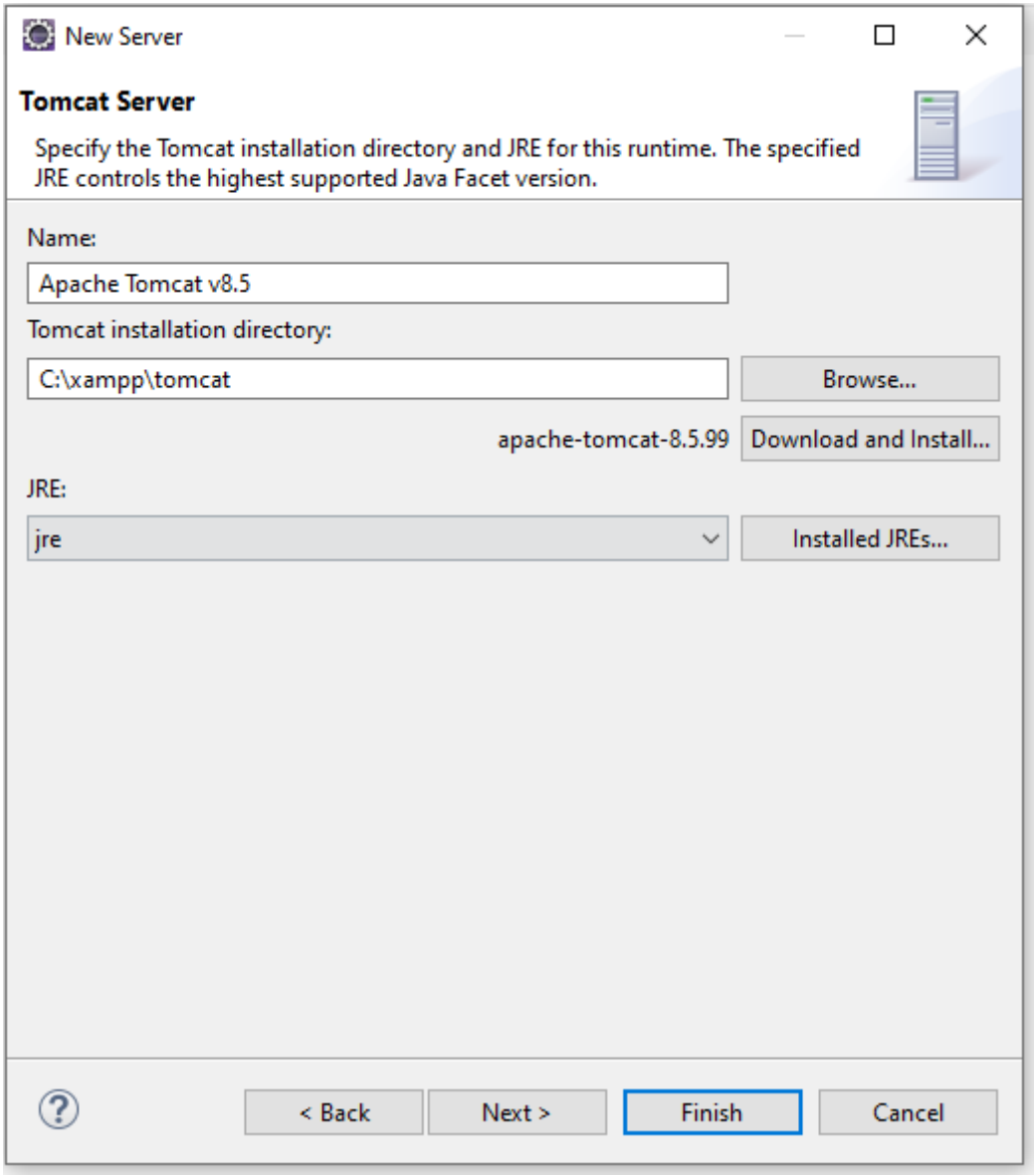
Comentario: Se muestra el asistente para crear o configurar un servidor en Eclipse. Aquí se selecciona el tipo de servidor que se utilizará para ejecutar la aplicación.

Definiendo servidor



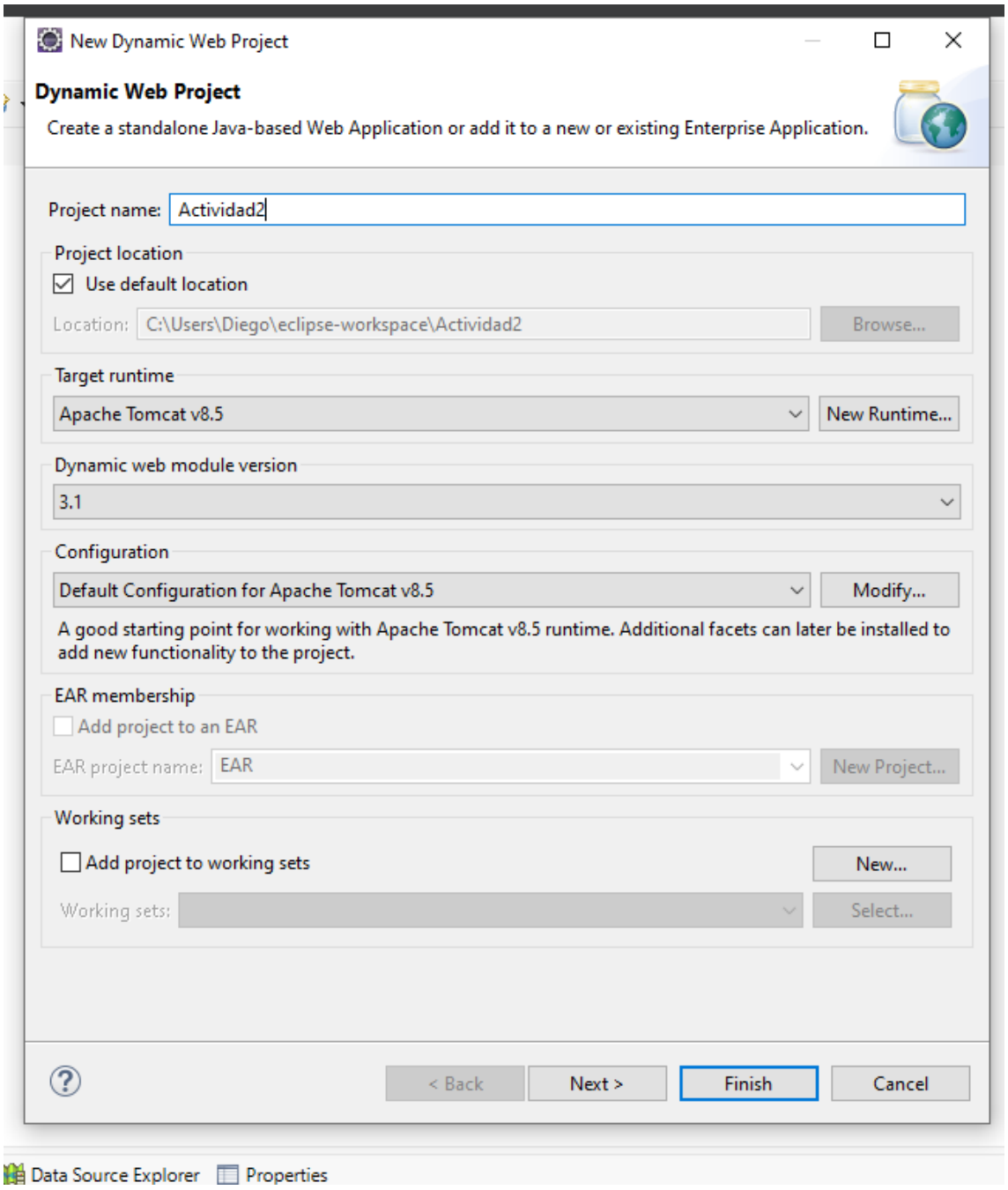
Comentario: En esta pantalla se elige Apache Tomcat como servidor y se indica su instalación local. Después se puede finalizar la configuración.

Tomcat



Comentario: La imagen muestra la configuración del proyecto web dinámico. Se define el nombre del proyecto y las opciones necesarias para crearlo en Eclipse.

Definimos proyecto



Comentario: Una vez creado el proyecto, aparece en Eclipse la estructura de carpetas y archivos. Desde aquí se pueden añadir páginas web y código Java.

Codigo

MyServlet.java

```java

import java.io.IOException;

/**
 * Servlet implementation class MyServlet
 */
@WebServlet("/MyServlet")
public class MyServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    /**
     * @see HttpServlet#HttpServlet()
     */
    public MyServlet() {
        super();
        // TODO Auto-generated constructor stub
    }

    /**
     * @see HttpServlet#doGet(HttpServletRequest request, HttpServletResponse response)
     */
    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
        // TODO Auto-generated method stub
        response.getWriter().append("Served at: ").append(request.getContextPath());
    }

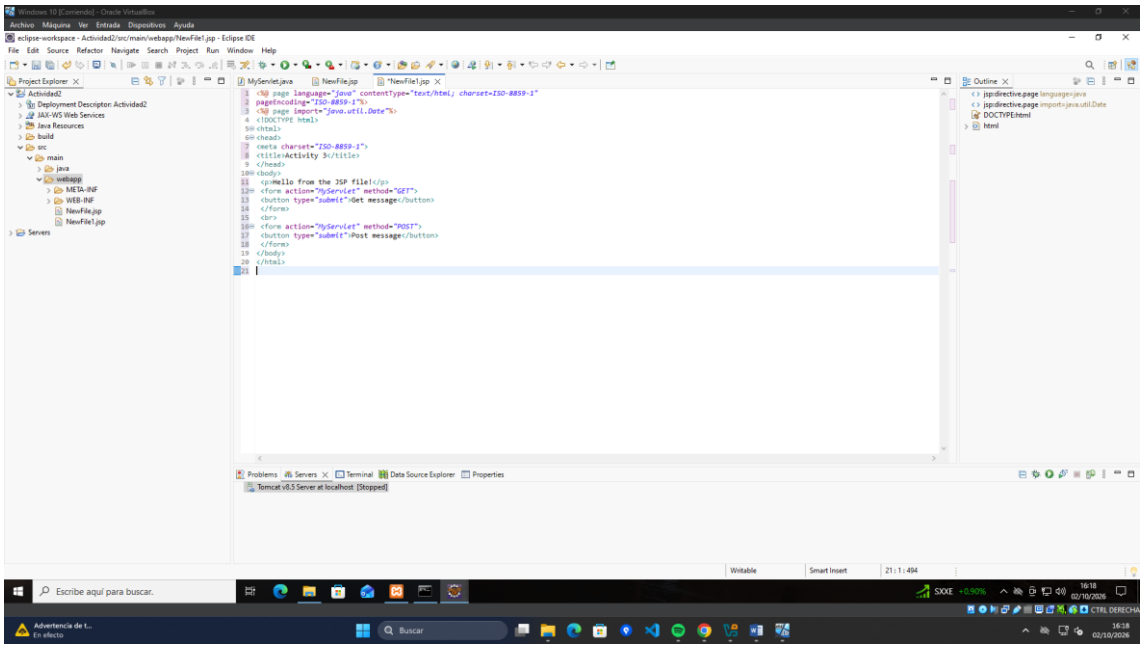
    /**
     * @see HttpServlet#doPost(HttpServletRequest request, HttpServletResponse response)
     */
    protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
        // TODO Auto-generated method stub
        doGet(request, response);
    }

}

```

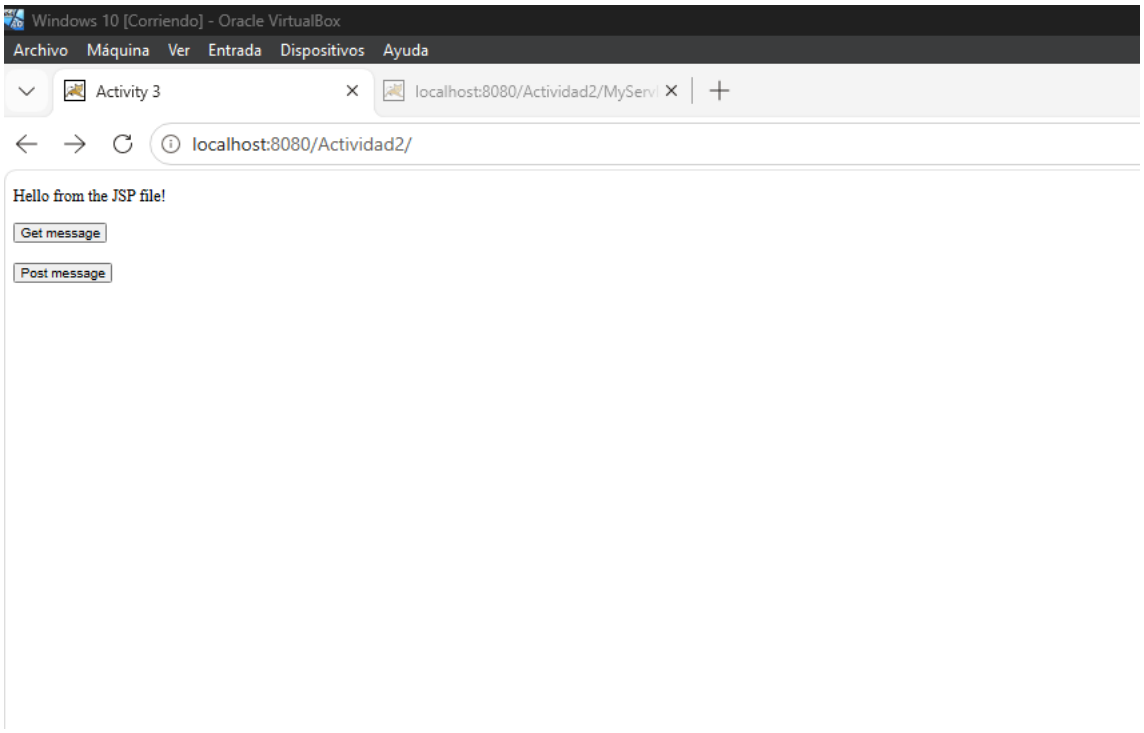
Comentario: En esta imagen se observa el código del servlet Java. El servlet recibe el número del DNI, comprueba que sea válido y calcula la letra correspondiente.

Comentario: Se muestra el entorno de Eclipse con el proyecto y sus archivos. Desde esta vista se puede editar el código y revisar los errores del proyecto.

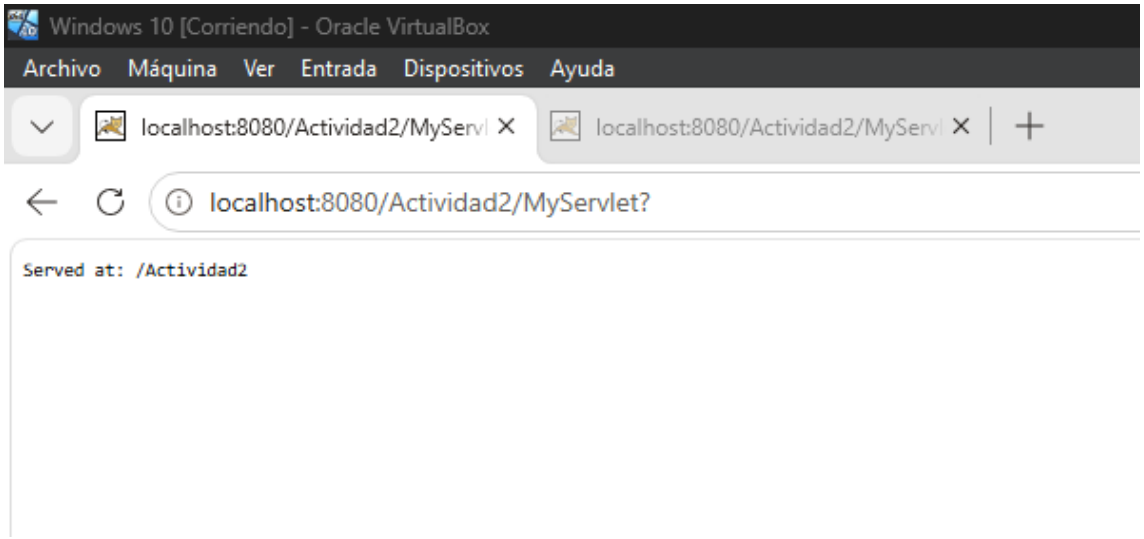


Comentario: Esta captura muestra la configuración del servidor Tomcat dentro de Eclipse. Sirve para revisar los datos del servidor antes de iniciar la aplicación.

Resultados

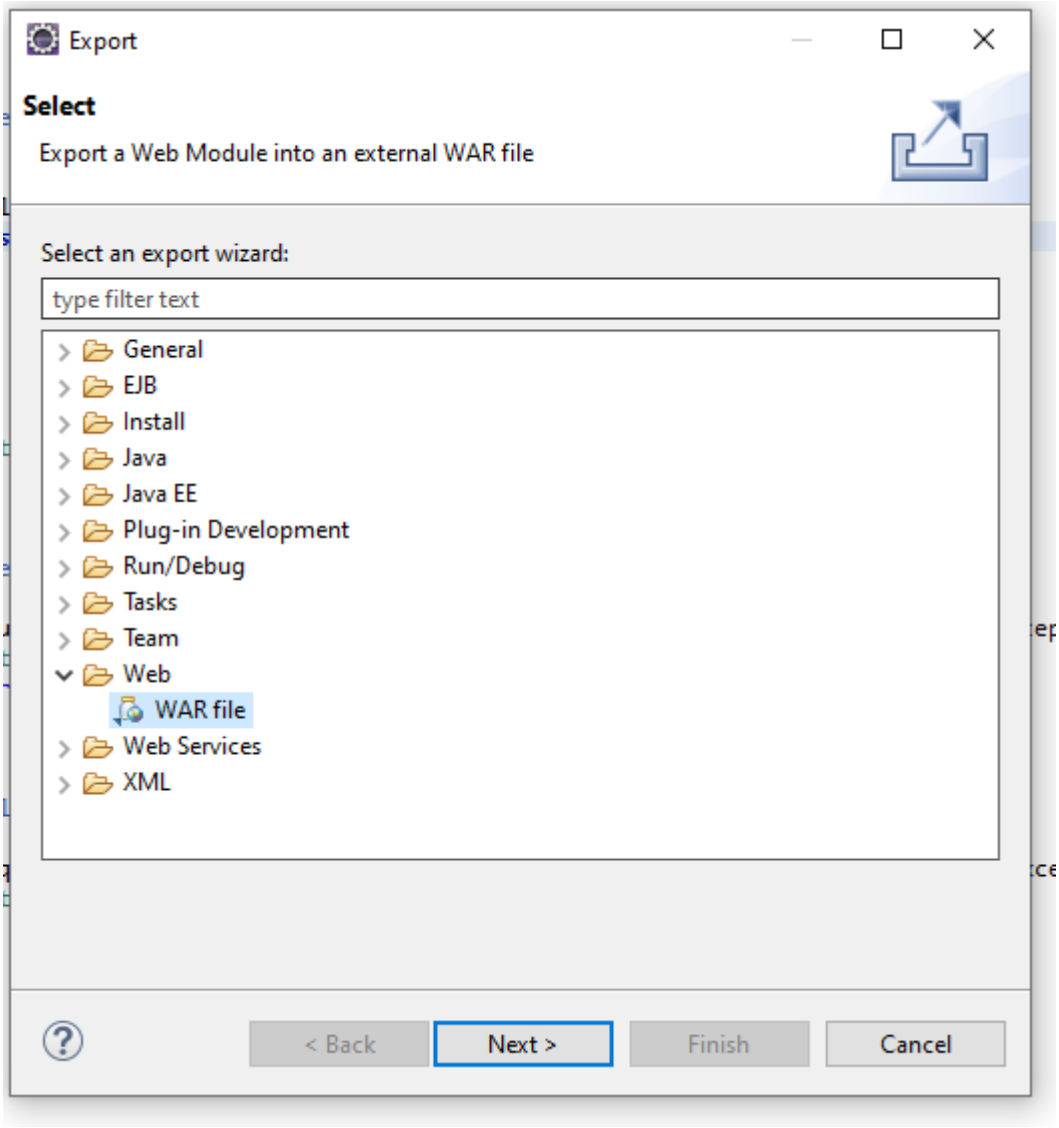


Comentario: Aquí se ve la página web ejecutándose en el navegador local. El formulario permite introducir los números del DNI y pulsar el botón para calcular la letra.



Comentario: La otra captura corresponde al asistente de exportación de Eclipse. Este menú permite elegir el tipo de archivo o proyecto que se quiere exportar.

Archivo WAR



Comentario: Esta imagen muestra el resultado o la página de la aplicación abierta en el navegador, accediendo a través de localhost. Permite comprobar que el servidor está respondiendo.

```
package servlets;

import java.io.IOException;
import java.io.PrintWriter;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@WebServlet("/LetraDNIServlet")
public class LetraDNIServlet extends HttpServlet {

    private static final long serialVersionUID = 1L;
    private static final String LETRAS = "TRWAGMYFPDXBNJZSQVHLCKE";

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse
response)
            throws ServletException, IOException {
        mostrarPagina(response, null, null);
    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse
response)
            throws ServletException, IOException {
```

```
        request.setCharacterEncoding("UTF-8");
        String entrada = request.getParameter("dni");
        String resultado;
        String error = null;

        try {
            int dni = Integer.parseInt(entrada.trim());

            if (dni < 0 || dni > 99999999) {
                error = "El DNI debe ser positivo y tener como máximo 8
dígitos.";
                resultado = null;
            } else {
                char letra = LETRAS.charAt(dni % 23);
                resultado = String.format("%08d-%c", dni, letra);
            }
        } catch (NumberFormatException | NullPointerException e) {
            error = "Introduce solo números.";
            resultado = null;
        }

        mostrarPagina(response, resultado, error);
    }

    private void mostrarPagina(HttpServletResponse response, String
resultado, String error)
            throws IOException {

        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();

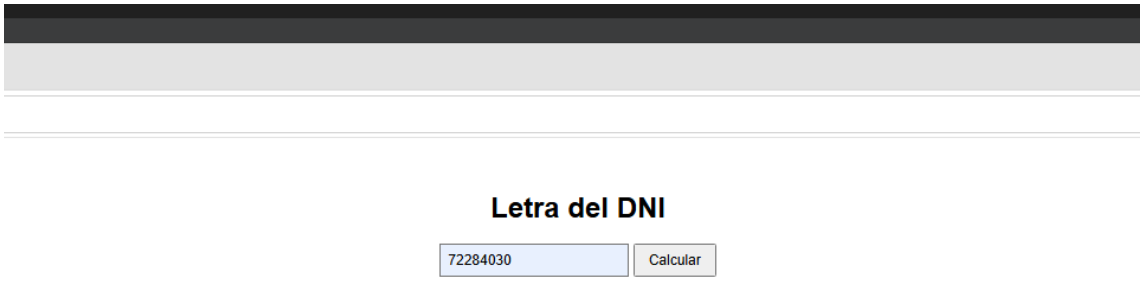
        out.println("<!DOCTYPE html>");
        out.println("<html lang='es'>");
        out.println("<head>");
        out.println("<meta charset='UTF-8'>");
        out.println("<title>Calculadora de letra del DNI</title>");
        out.println("<style>");
        out.println("body{font-family:Arial,sans-serif;max-
width:400px;margin:60px auto;text-align:center}");
        out.println("input{padding:8px;font-size:16px}");
        out.println("button{padding:8px 16px;font-
size:16px;cursor:pointer}");
        out.println(".ok{color:#1b7f3b;font-size:22px;margin-top:20px}");
        out.println(".error{color:#c0392b;margin-top:20px}");
        out.println("</style>");
        out.println("</head>");
        out.println("<body>");
        out.println("<h1>Letra del DNI</h1>");
```

```
        out.println("<form method='post' action='LetraDNIServlet'>");
        out.println("<input type='text' name='dni' maxlength='8'
placeholder='8 números' required>");
        out.println("<button type='submit'>Calcular</button>");
        out.println("</form>");

        if (resultado != null) {
            out.println("<p class='ok'>Tu DNI completo es: <strong>" +
resultado + "</strong></p>");
        }
        if (error != null) {
            out.println("<p class='error'>" + error + "</p>");
        }

        out.println("</body>");
        out.println("</html>");
    }
```

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Calculadora de letra del DNI</title>
    <style>
        body { font-family: Arial, sans-serif; max-width: 400px; margin: 60px
auto; text-align: center; }
        input { padding: 8px; font-size: 16px; }
        button { padding: 8px 16px; font-size: 16px; cursor: pointer; }
    </style>
</head>
<body>
    <h1>Letra del DNI</h1>
    <form method="post" action="<%= request.getContextPath()
%>/LetraDNIServlet">
        <input type="text" name="dni" maxlength="8" pattern="[0-9]{1,8}"
placeholder="8 números" required>
        <button type="submit">Calcular</button>
    </form>
</body>
</html>
```



Comentario: Esta última captura muestra la página de la calculadora de la letra del DNI. El usuario introduce los números y pulsa «Calcular» para enviar el formulario al servlet.

Preguntas de teoría:

Investiga sobre los siguientes temas y proporciona una respuesta corta (no más de 50 palabras cada uno):

a. ¿Cuál fue la parte más compleja de esta actividad? ¿Cómo lograste resolverla?

   La parte más compleja fue configurar Eclipse con Tomcat y conseguir que el servlet funcionara correctamente. Lo resolví siguiendo los pasos de instalación, revisando el código y comprobando que el servidor arrancara bien.

b. ¿Puedes encontrar otro servidor de aplicaciones y compararlo con Tomcat?

   Otro servidor de aplicaciones es WildFly. Ambos permiten ejecutar aplicaciones Java, pero tienen algunas diferencias. Tomcat está más centrado en aplicaciones web que utilizan servlets y JSP, mientras que WildFly ofrece más funciones de Java empresarial, como la gestión de transacciones y otros servicios avanzados. Por eso, Tomcat puede resultar más sencillo para proyectos web básicos, mientras que WildFly está preparado para aplicaciones empresariales más completas.